

Deliverable #2

CometContact

Group Members

- Adrian Alvarez
- Shaikh Hamid
- Isioma Emordi
- Nicole Maing
- Anirudh Vayalali
- Ethan Varghese
- Joshua Gambrell

Deliverable # 2 [Each member's tasks]

- Adrian Alvarez: I will be writing the project scheduling, cost estimation, and editing the Presentation Recording
- Anirudh Vayalali: I will be comparing our work to other similar designs based on some common categories that I will decide after doing the thorough research (#6 in the Deliverable Specification) and ensuring the GitHub repository is clean and well-kept for project submission.
- Shaikh Hamid: I will be writing the conclusion for the group project. I will be honest and make sure to list the lessons learned throughout the semester.
- Isioma Emordi: I will write a test plan for testing one unit of our software and test using an automated testing tool
- Nicole Maing: I will be organizing and designing our presentation slides and gather all references used in our project.
- Joshua Gambrell: I will keep track of the delegation list or who is doing what.

GITHUB: <https://github.com/axdyv/3354-CometContact>

TASK REPORT:

3. Deliverable #1 Items

a. Assumptions

This application will be developed assuming the university agrees to integrate certain administrative systems with the authorization and security features of the application. The system would have to integrate with key UTD resources such as the UTD Coursebook and use UTD's Single-Sign On system to verify user enrollment and ensure security.

Additionally, the application will follow FERPA and other data regulation guidelines/legislation based on the country of the United States and specifically, the state of Texas.

b. Feedback from Proposal

No constructive feedback to be gained from the proposal (All that was written was 'Good Job')

c. Software Process Model

The *Incremental Process Development Model* would be the most appropriate for CometContact because it allows us to deliver working software while being able to build upon it with flexible requirements. Since the software is also going to be heavy on the features, starting with a strong base with the core functionality will allow students and faculty to interact with the software and provide feedback. Features that we may have not thought about could be a commonly requested item and, thus, we will be able to implement it in following increments. This process model also allows us to plan out our increments and prioritize the features that either we believe are the most necessary or the users request. The only disadvantage that we will have to be careful of is to not let our structure degrade with every new increment. Making sure that the core of our software is reliable and easily maintainable is an important factor to make this process model work.

The *Waterfall Model* will not work because of its rigid structure and lack of user involvement. Our project will rely on users to provide much-needed feedback and this process model lacks flexibility. The *V-Model* is similar to the *Waterfall Model* in that it has a rigid structure, especially with the requirements phase. Therefore, it would also not be a good option for CometContact because of the same reasons. The *Prototyping Model* is an appealing model for its iterative and user-involved development. However, it may be difficult to maintain and scale in the long run if the development process becomes unstructured. It would be more beneficial to get the software, even in a basic form, out to users to utilize and test than to build upon prototypes before releasing a final version. Finally, the *Spiral Model* is a bit of overkill for a simple application that would be integrated into eLearning. If this was a large-scale standalone software with high risk, then this process model would be an appealing one. However, in our case, it would only add to the overhead, increase complexity, and slow down our development.

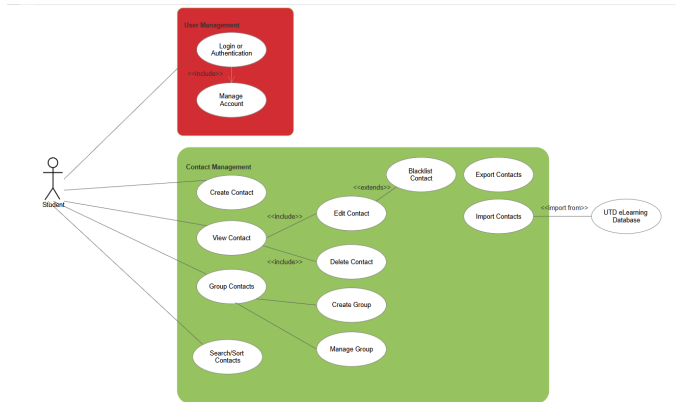
d. Software Requirements

Starting with the functional requirements for our project, CometContact, the user will be able to find classmates by certain attributes. By using coursebook and UTD eLearning, the system will allow a student to search and filter classmates by course, section, name, and NetID. Next, the system will allow the user to create and join groups for courses. It will allow users to create

groups with capacity limits set by the creator. Additionally, the system will allow users to add, delete. Finding students can be done through the filter system and adding/deleting contacts will be done by the system as well. The system will also allow the user to update their own profile. Certain attributes that are based on the SSO login wouldn't be changeable (such as NetID) but the user can change/add things like preferred name or profile picture. In order to maintain security, the system will also allow the user to block/unblock contacts. By giving users the ability to block contacts, the system puts an extra layer of security in the hands of the user. Last but certainly not least, the system will verify user status by using the school's SSO login and ensuring that the user is enrolled as a student and limit the selection of classes based on their current term registration. This would be done by being connected to the coursebook/schedule planner school administrative system. As each of these requirements are processed, we are aiming to prioritize security and safety in order to ensure there are no rogue users or random people joining the system.

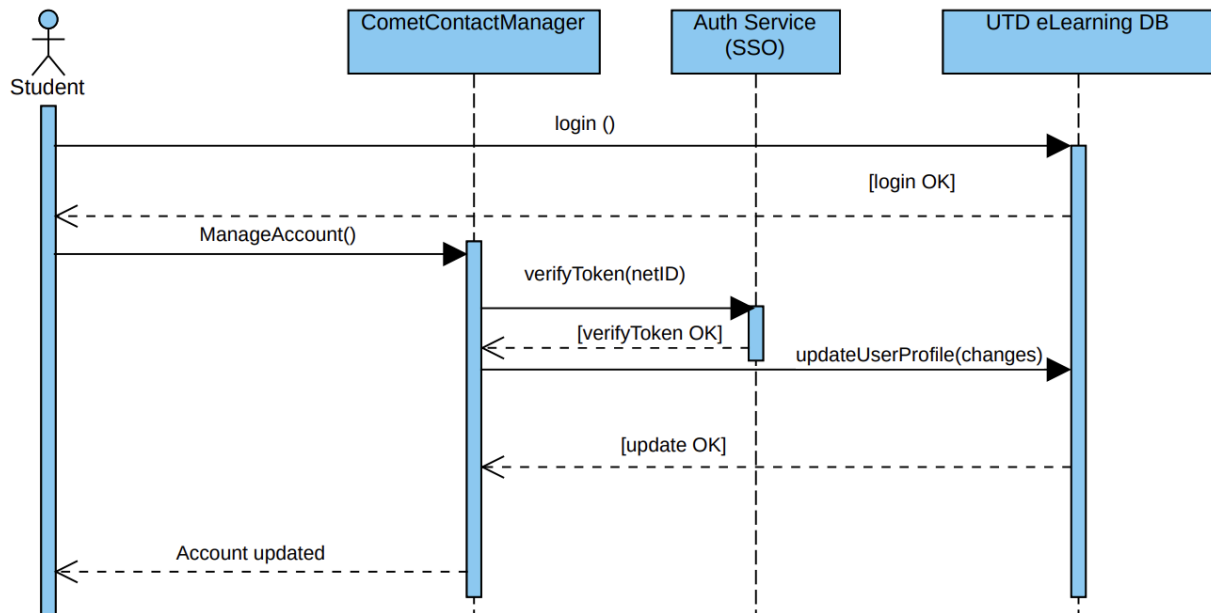
For the non-functional requirements, the product will need to be performance efficient given the vast number of UTD students and large number of notifications. The next most important product requirement would have to be the reliability of the service, making sure the system is active during the most important parts of the semester (enrollment times, exam seasons, project deadlines, etc.) The last product requirement that would be helpful to have is a scalable and maintainable codebase. Given the growing number of UTD students, having a scalable and easily maintainable codebase is increasingly important, especially for a product that requires such a high level of reliability and usability. In terms of organizational requirements, the development will use staged environments to help with our incremental development process model. The next requirement would be ensuring the standards/security of our application fall in-line with university secure coding standards. The techstack of such an application would have to be university approved in order to request the integration with existing university administrative applications. Lastly, for external requirements, we would have to place a greater importance on things like ethics and safety. One of the issues with making an application that helps connect people, is the chance that some people may use it to take advantage of others in a negative way. Having a way to block contacts and enforce campus security online is incredibly important. In addition to this, the application would have to follow FERPA guidelines and enforce the non-disclosure of non-directory information without consent, especially if we allow users to export/import contacts.

e. Use Case Diagram

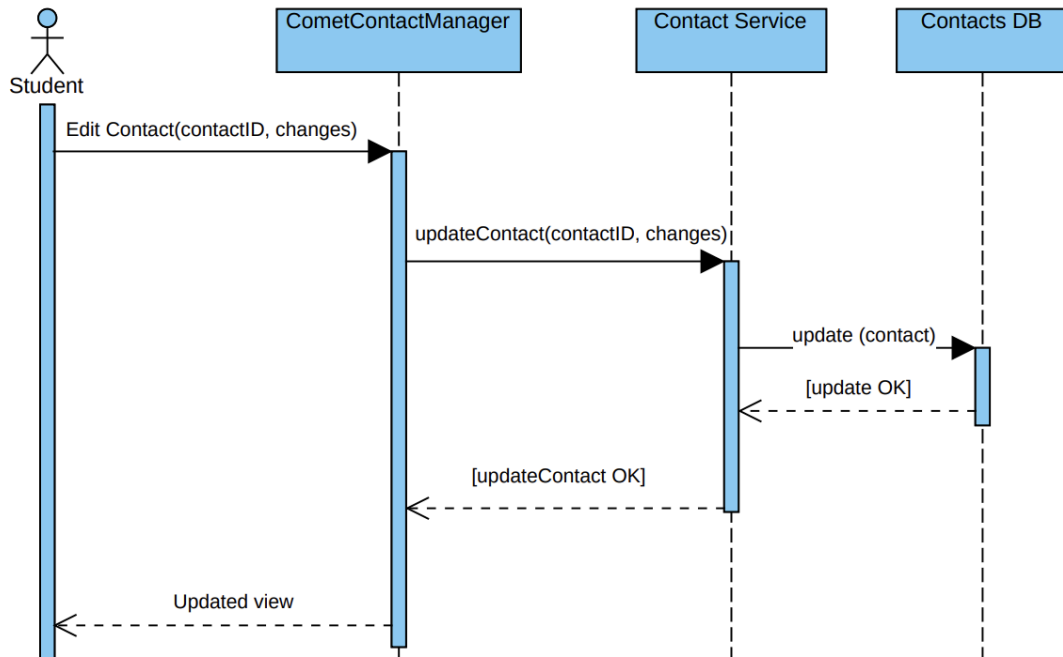


f. Sequence Diagrams

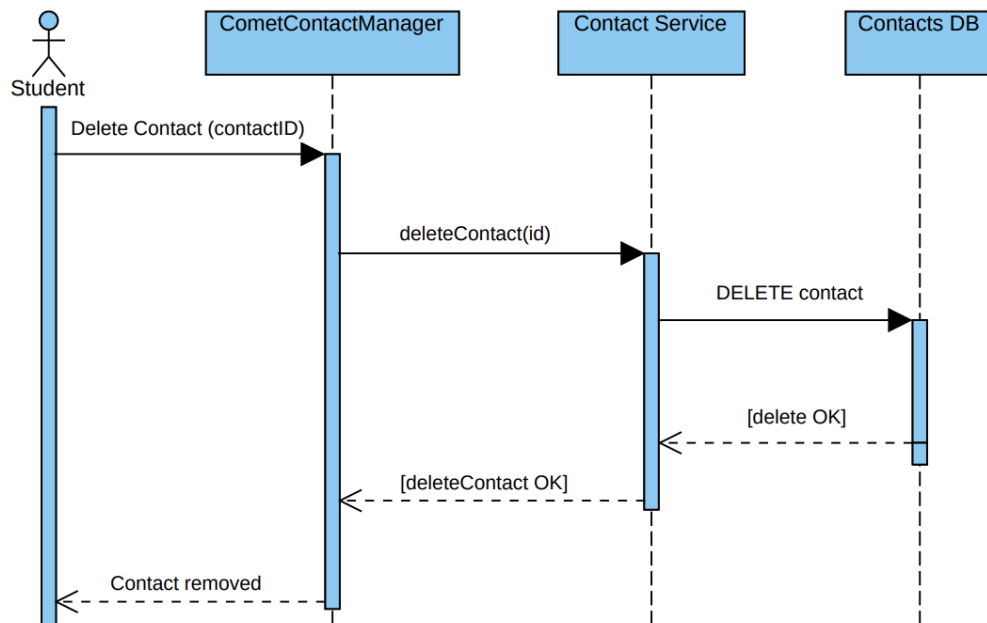
Contact Manager:



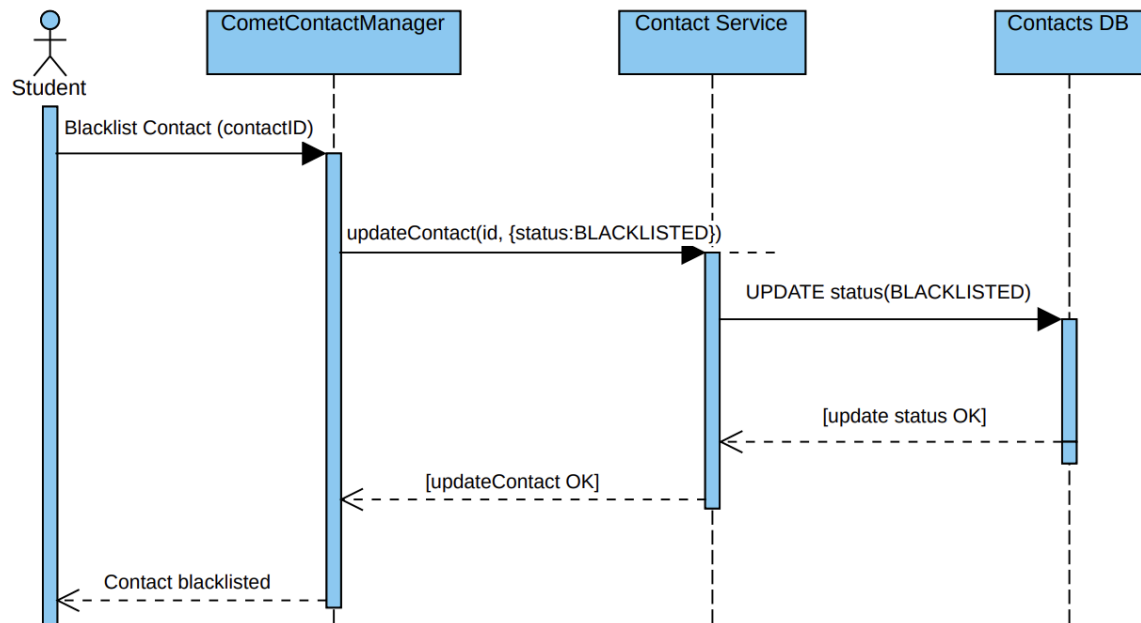
Edit Contact:



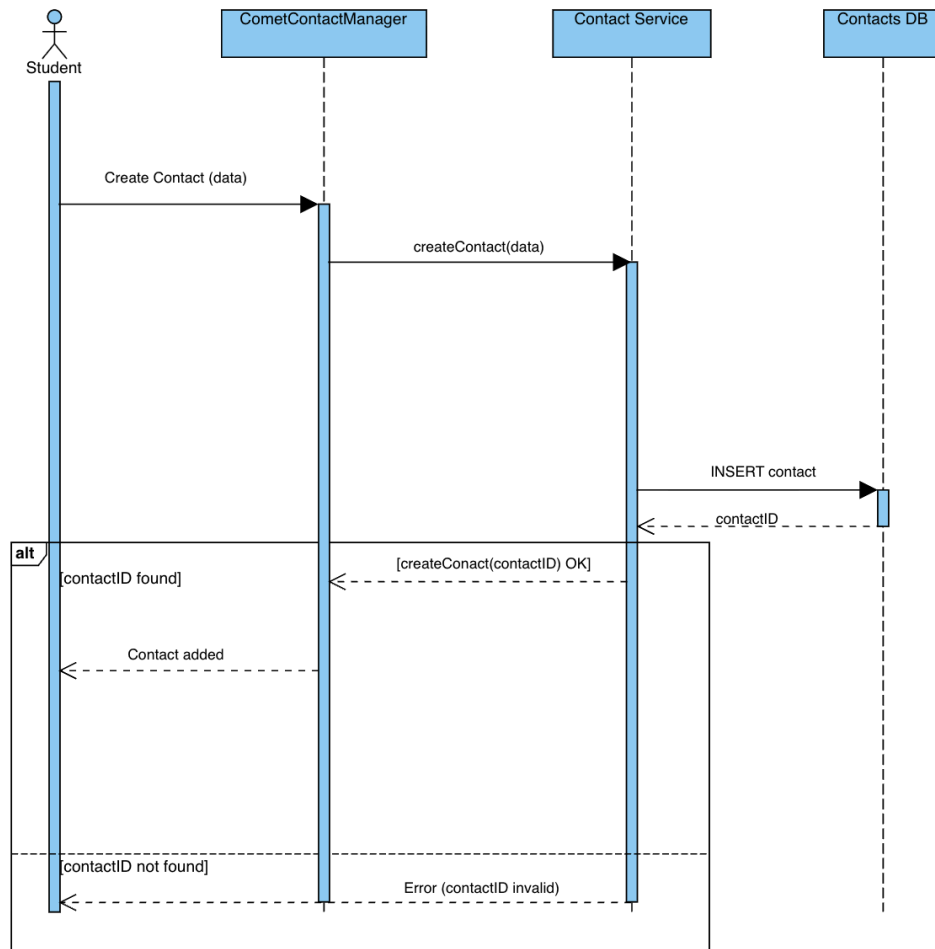
Delete Contact:



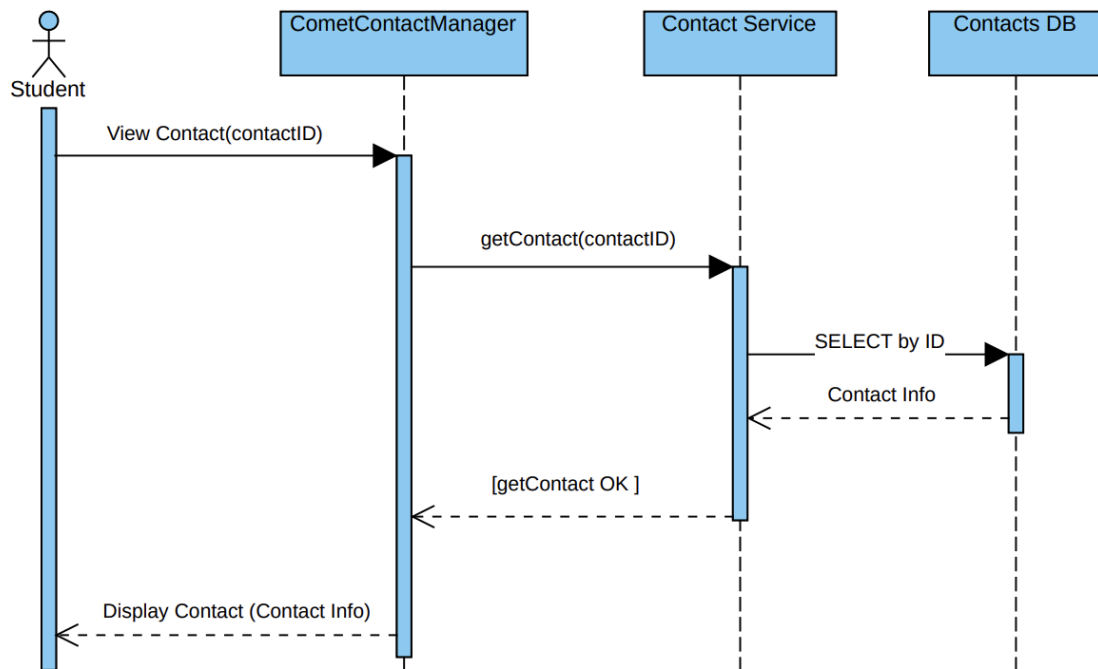
Blacklist (Block) Contact:



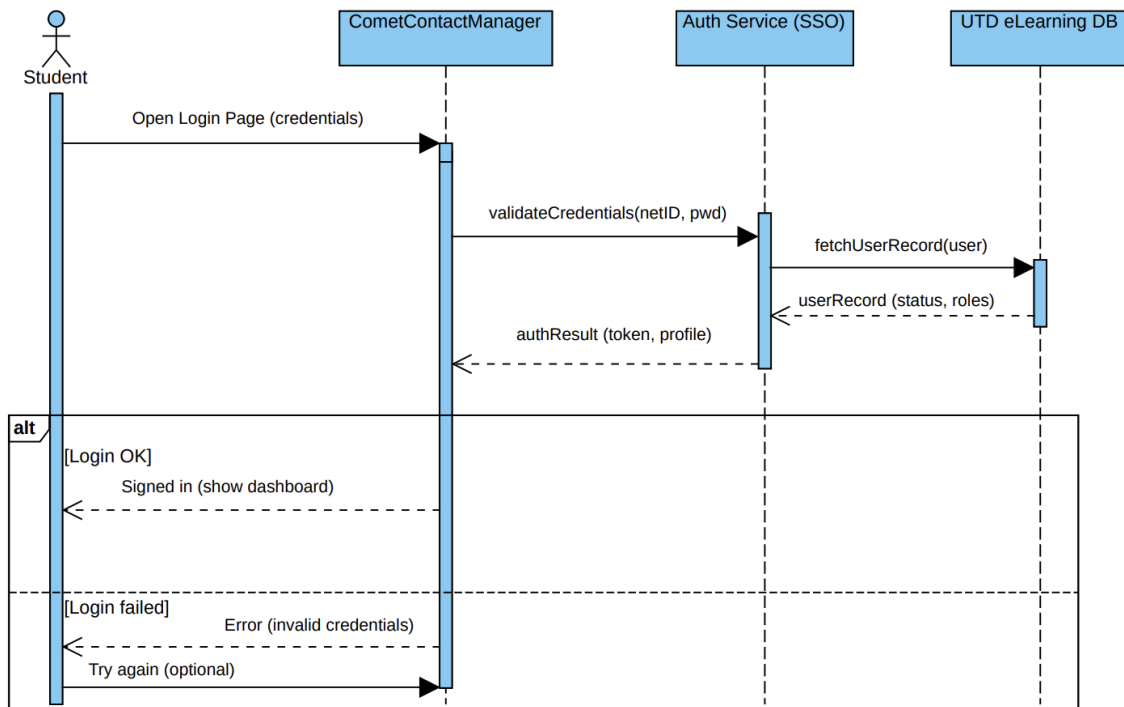
Create Contact



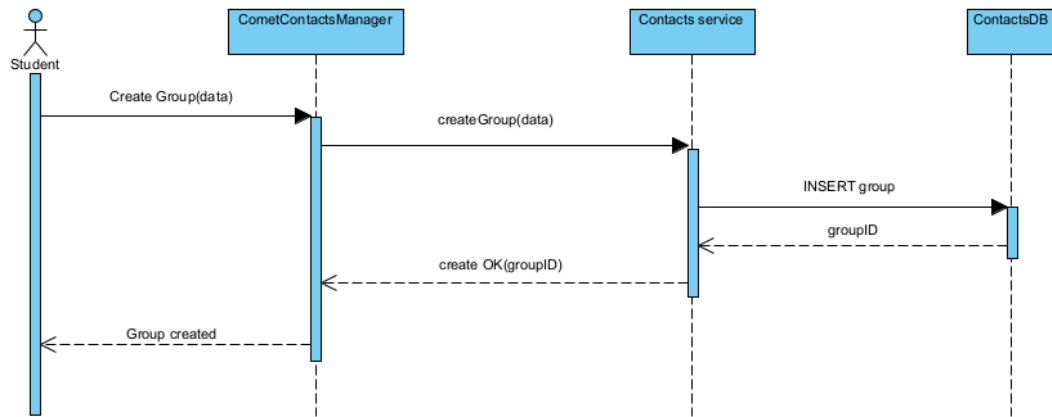
View Contact:



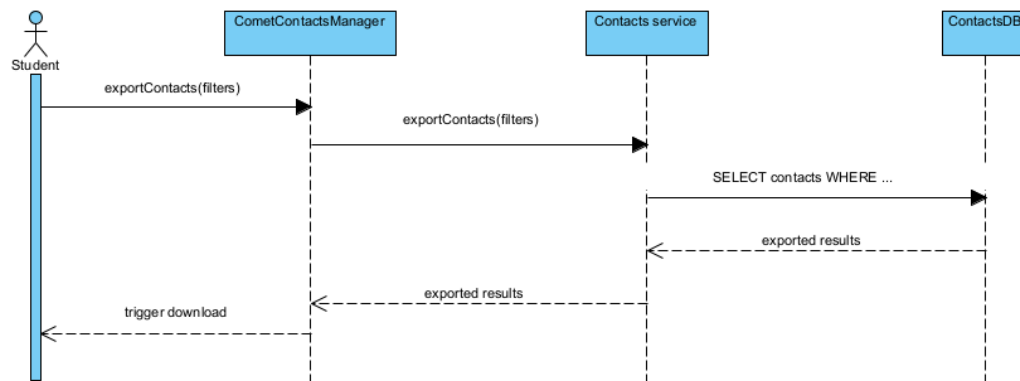
Authorization Sequence:



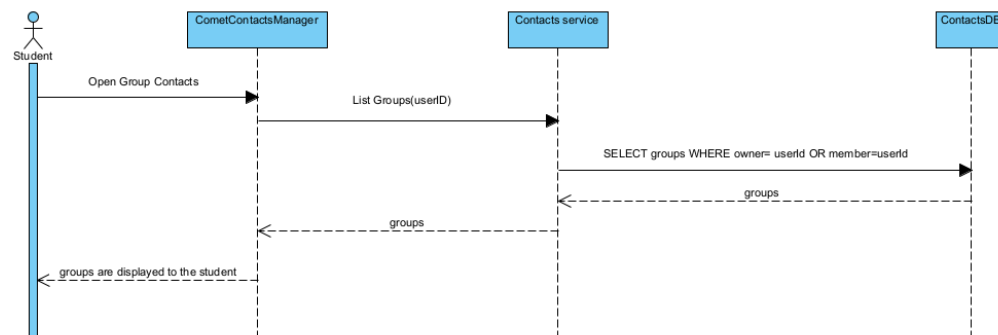
Create Group:



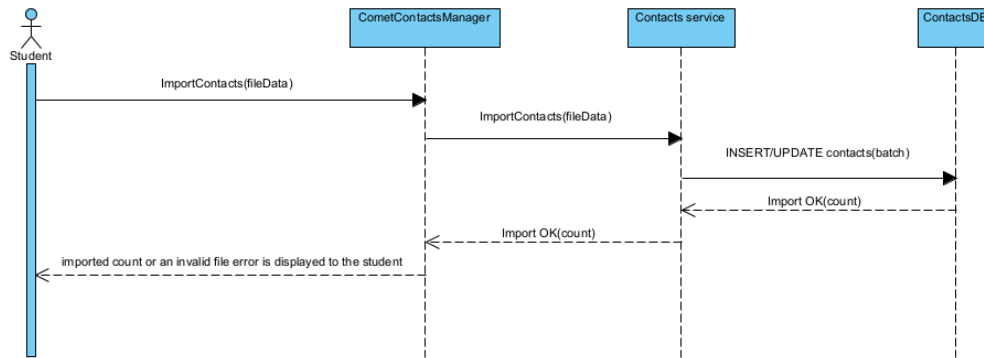
Export Contacts



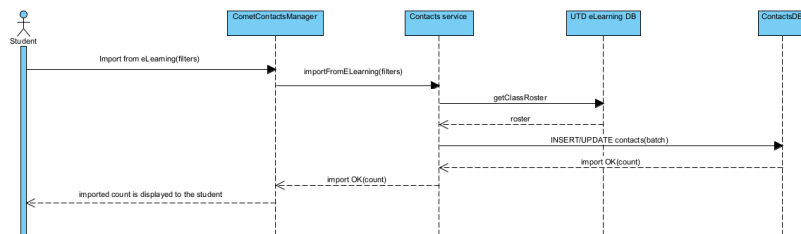
Group Contacts:



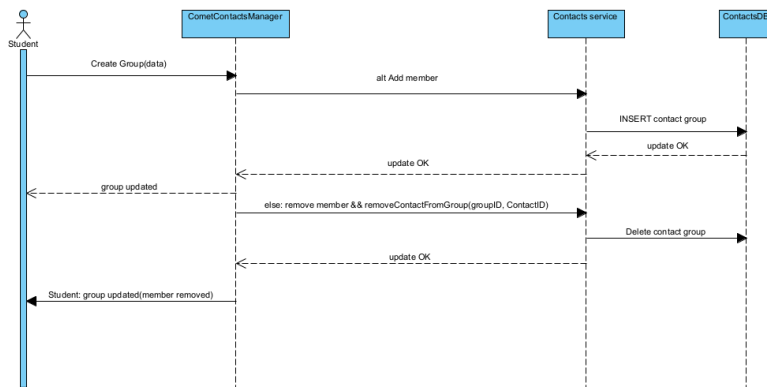
Import Contacts from File:



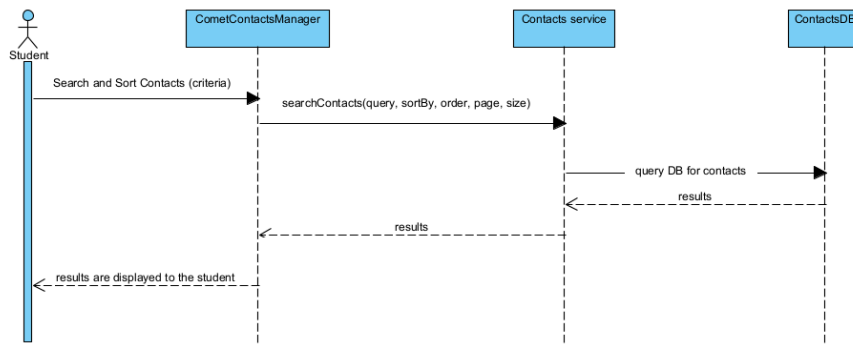
Import Contacts from eLearning:



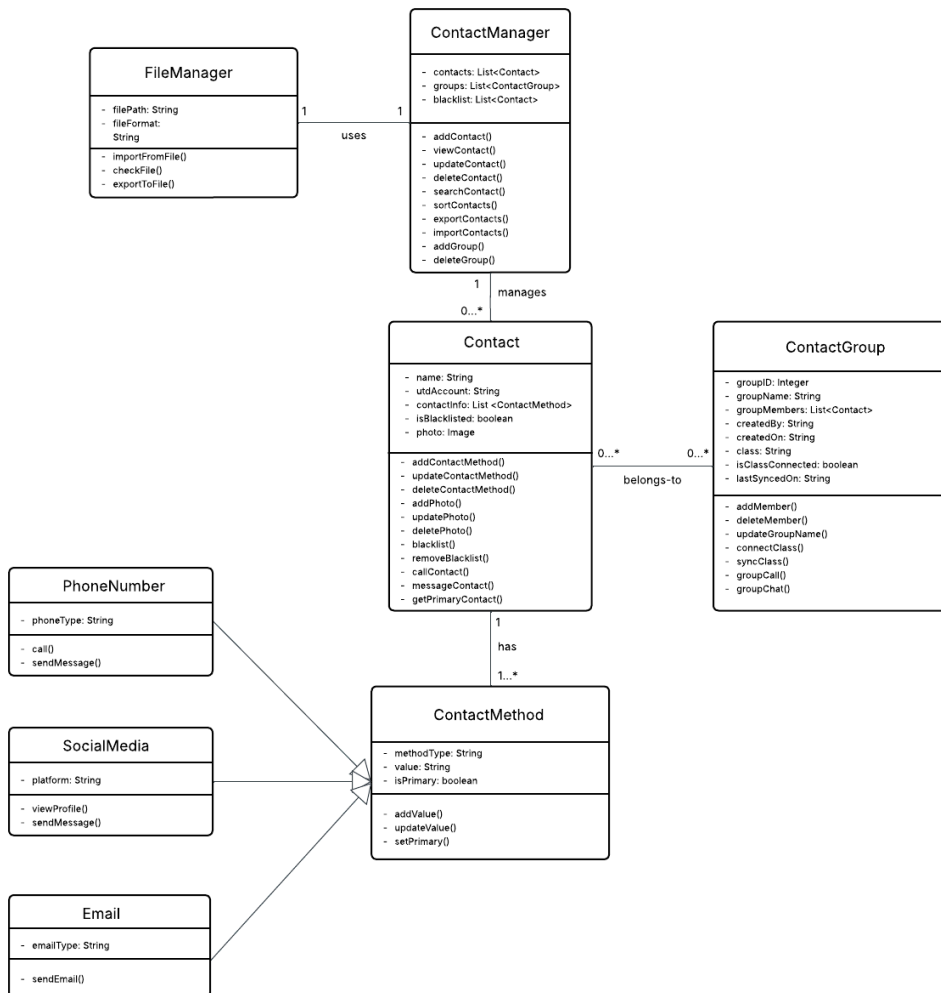
Manage Group:



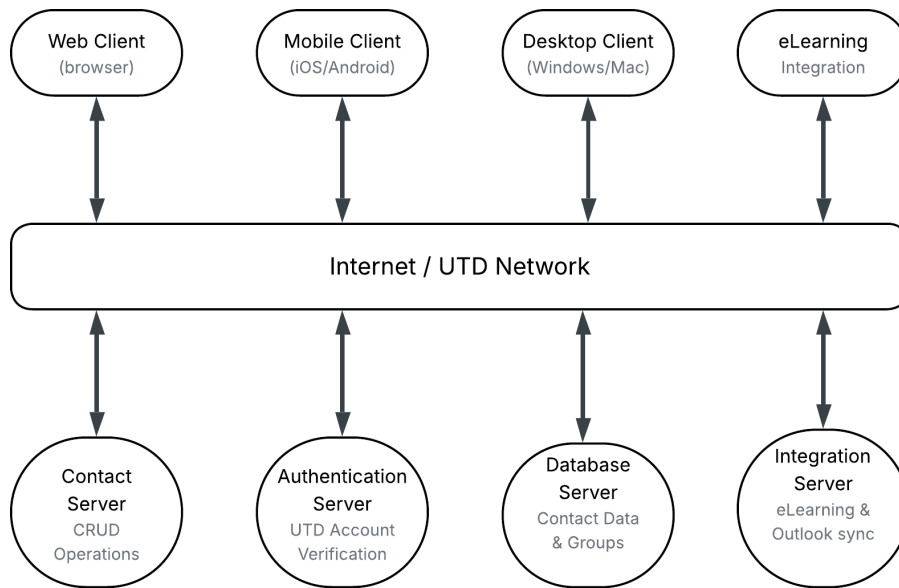
Search and Sort Contacts:



g. Class Diagram



h. Architectural Design



4. Project Planning

a. Project Scheduling

Assuming that there is a team of trained Software Engineers within the UT Dallas Office of Information Technology tasked with building CometConnect, this is how the project Planning will look (Using our Incremental Model and Agile Methodology)

Team: Full-Time Software Engineers

- 1 Project Manager, 1 UI/UX Engineer, 2 Backend Engineers, 2 Frontend Engineers, 1 QA/Test Engineer

8 Hour Working Days (No Weekends)

- 6 Hours of active development
- 1 Hour for Stand-Up
- 1 Hour for Administrative Tasks/Documentation

Agile Methodology (Scrum)

- 2-week Sprints

Dream Timeline:

- Sprint 1 (January 1st to January 14th)
 - Focus on requirement refinement
 - Database design
 - UTD SSO (NetID Auth) with UTD APIs
 - GitHub Repository Setup
- Sprint 2 (January 15th - January 28th)
 - Backend
 - API for Add/View/Edit/Delete contacts
 - Frontend

- Basic Dashboard and Contact List UI
- Feature
 - Manual Contact Creation
- Testing
- Sprint 3 (January 29th - February 11th)
 - Feature
 - Contact Groups (API for Create/Delete Groups)
 - Search Functionality
 - Blacklist Contacts
 - Testing
- Sprint 4 (February 12th - February 25th)
 - Feature
 - Direct Call and SMS
 - Support multiple phone and social accounts
 - Sorting (Name/Group)
 - Testing
- Sprint 5 (February 26th - March 10th)
 - Feature
 - Automated Class List from UTD/eLearning
 - Auto-grouping based on classes
 - Testing
- Sprint 6 (March 11th - March 24th)
 - Feature
 - Import/Export Contact List
 - Photo Upload for Contact
 - Frontend
 - UI Polish
 - Testing
- Sprint 7 (March 25th - April 5th)
 - Integration Testing
 - User Testing
 - Bug Fixes
 - Deployment

We chose our Start Date to be January 1st because it is the start of the new year after the holidays. April 5th became our rough estimate for the End Date of our project because it lined up with how we planned to spread out our functionality development across the sprints. We only needed about 5-6 Sprints for developing the program's functionalities, but dedicated the last sprint (Sprint 7) for purely testing and deployment.

b. Method of Cost/Effort/Pricing Estimation

We used the Function Point Method because it helps us quantify what our program will do for the user. Since our program is heavily 'Features' based, it is more accurate to make each function a fixed amount of points. Since we rely on integrations with external UTD systems, we are able to use the Function Point Method to weigh the complexities and provide us a more realistic estimation than. We decided not to use Application Composition (COCOMO II) because CometContact is feature-heavy. There isn't too many graphical user interfaces as much of the complexity is in the backend.

Gross Function Point Estimation:

- 5 FP - External Inputs: Add/Edit/Delete/Import Contacts/Groups, Photo Label Feature
 - Complexity: 4X Weight for Core Functionality = 20 FPs
- 3 FP - External Outputs: Export, Call, SMS
 - Complexity: 4X Weight for Core Functionality = 12 FPs
- 3 FP - External Inquiries: Search, View, Sort Contacts
 - Complexity: 3X Weight for Basic Functionality = 9 FPs
- 3 FP - Internal Logical Files: Contact Database, Grouping Database, User Settings
 - Complexity: 10X Weight for Database Implementation = 30 FPs
- 2 FP - External Interface Files, UTD SSO, UTD Registrar
 - Complexity: 10X Weight for UTD Integration = 20 FPs
- Total = 91 GFP

Processing Complexity (Based on the 14 Question for Processing Complexity):

1. 5 - Web-Based, Dependent on API calls, Very reliant on backup and recovery
 2. 3 - Client-Server Architecture
 3. 3 - Requires standard response times
 4. 4 - Must run on multiple platforms
 5. 3 - Moderate use by students
 6. 5 - All contacts are entered and edited in the program
 7. 4 - UI must be friendly
 8. 5 - Data must update in real-time online
 9. 3 - Basic inputs for contacts and files with contacts
 10. 1 - Auto-Mapping UTD classes with contacts
 11. 3 - Low-reuse, UTD Specific
 12. 3 - Normal conversion and installation
 13. 1 - UTD Only
 14. 3 - Agile methodology implies ease of change and use
- Total = 46 PC

PC Adjustment

- $PCA = 0.65 + (0.01 * PC) = 0.65 + (0.01 * 46) = 0.65 + 0.46 = 1.11$

Function Point Estimation:

- $81 * 1.11 = 101.11 \text{ FP} \rightarrow \sim 101 \text{ Function Points}$

c. Estimated Price of Hardware Products

Using: Cloud Virtual Machines for Backend and Database, Servers

- 2 App Servers for Dev and Staging (AWS Servers) = ~\$60.00 per Month
- Database (AWS w/ SQL) = ~\$80.00 per Month
- File Storage (AWS) = ~\$10.00 per Month

Total per Month = \$150 per Month → About 4 Months → \$600 Total

d. Estimated Price of Software Products

Using: Free Open-Source Frameworks, University Integrations, VS Code, Project Management, GitHub

- IDE (VSCode or similar) + Open-Source Frameworks + UTD Systems = Free
- Project Management (Jira or Similar) = \$8.50 per User per Month
- Documentation (Confluence) = \$6.00 per User per Month
- GitHub Actions = \$40.00
- Design Tools (Figma or Similar) = \$15.00 per User per Month

Total per Month = \$157.00 → About 4 Months → \$628 Total

e. Estimated Price of Personnel

For 1 Engineers in the DFW Area: \$110,041/year [1] = \$9170 per Month

For 6 Engineers: \$55,021 per Month

f. Final Estimations

Function Points: 101 FP

Productivity Rate: ~ 4 FP/Person-Month → $101/4 = 20.2$ Person-Months

Engineers: 6 → $20.2 \text{ Person-Months} / 6 \text{ Engineers} = 3.36$ Months

Labor Cost: $3.36 * 55,021 = \sim \$185,000$

Total Cost: $\sim \$185,000$ (Labor) + \$628 (Software) + \$600 (Hardware) = \$186,228

5. Test Plan

a. Test Plan for One Unit of Software

Unit: `ContactManager.addContact(Contact c)`

This method adds a new Contact object to the user's contact list and checks:

1. The contact to be added is not a duplicate.
2. The required fields are filled (NetID).
3. The returned value reflects success or failure.

Objectives:

- Verify correct addition of a valid contact.
- Verify that duplicates are rejected.
- Verify that invalid contacts (e.g., missing NetID) are rejected.
- Verify that the internal data structure updates correctly.

Method:

- Use JUnit automated testing to run independent, repeatable tests.
- Use black-box strategies - provide input and verify expected output with equivalence partitioning.

Pass/Fail Criteria:

A test case passes if the method returns expected boolean status and the internal contact list reflects the expected change.

b. Code for One Unit of Software

```
import org.junit.Test;
import static org.junit.Assert.*;
public class AddContactTest {
    @Test
    public void testAddValidContact() {
        ContactManager cm = new ContactManager();
        Contact c = new Contact("Adam", "axe220000");
        boolean result = cm.addContact(c);
        assertTrue(result);
        assertEquals(1, cm.getContactCount());
    }
    @Test
    public void testAddDuplicateContact() {
        ContactManager cm = new ContactManager();
        Contact c1 = new Contact("Adam", "axe220000");
        Contact c2 = new Contact("Adam Dupe", "axe220000");
        cm.addContact(c1);
        boolean result = cm.addContact(c2);
        assertFalse(result);
        assertEquals(1, cm.getContactCount());
    }
    @Test
    public void testAddInvalidContact() {
        ContactManager cm = new ContactManager();
        Contact invalid = new Contact("Isi", null);
        boolean result = cm.addContact(invalid);
        assertFalse(result);
        assertEquals(0, cm.getContactCount());
    }
}
```

```
import java.util.HashSet;
public class ContactManager {
    private HashSet<Contact> contacts = new HashSet<>();
    public boolean addContact(Contact c) {
```

```

    if (c == null || c.getNetId() == null) {
        return false; // invalid
    }
    return contacts.add(c); // false if duplicate
}
public int getContactCount() {
    return contacts.size();
}
}

```

```

public class Contact {
    private String name;
    private String netId;
    public Contact(String name, String netId) {
        this.name = name;
        this.netId = netId;
    }
    public String getNetId() {
        return netId;
    }
    @Override
    public boolean equals(Object obj) {
        if (!(obj instanceof Contact)) return false;
        Contact other = (Contact) obj;
        return this.netId.equals(other.netId);
    }
    @Override
    public int hashCode() {
        return netId.hashCode();
    }
}

```

c. Test-Cases and Results

Test Case	Input	Expected Output	Actual Result
Add a new, valid contact	Contact("Adam", "axe220000")	Return true, contact count increased to 1	Returned true, contact count increased to 1
Add a duplicate contact	Contact("Adam", "axe220000") Contact("Adam Dupe", "axe220000") 2 contacts with same NetID	Return false, contact count remains 1	Returned false, contact count remained 1

Add an invalid contact	Contact("Isi", null) No NetID	Return false, contact count remains 0	Returned false, contact count remain 0
------------------------	----------------------------------	---	--

The screenshot shows the Eclipse IDE interface. The main editor displays the `AddContactTest.java` file with the following code:

```

16
17
18 @Test
19 public void testAddDuplicateContact() {
20     ContactManager cm = new ContactManager();
21     Contact c1 = new Contact("Adam", "axe220000");
22     Contact c2 = new Contact("Adam Dupe", "axe220000");
23
24     cm.addContact(c1);
25     boolean result = cm.addContact(c2);
26
27     assertFalse(result);
28     assertEquals(1, cm.getContactCount());
29 }
30
31 @Test
32 public void testAddInvalidContact() {
33     ContactManager cm = new ContactManager();
34     Contact invalid = new Contact("Isi", null);
35
36     boolean result = cm.addContact(invalid);

```

The left sidebar shows the Package Explorer with the `AddContactTest` class selected. The right sidebar shows the Outline view with the same class selected. The bottom status bar indicates 0 errors, 1 warning, and 0 others. The warning is: "The value of the field Contact.name is not used" in `Contact.java` at line 2.

6. Comparing to Similar Projects

When comparing CometContact to existing solutions, we looked at tools that helped connect students in similar ways to CometContact. One of the first ones that came to mind was Piazza, which is an online Q&A platform that creates a central discussion space for each course. This allows students to get quick and relevant guidance directly from their instructors and features an upvote rating system, ensuring that important questions can be voted on by students and seen quicker by professors. Piazza does allow students to communicate indirectly through these Q&A threads, but lacks the direct connection we aim to have with CometContact.

Another similar project is Campuswire, which is a class communication platform with a modern Q&A forum, chat rooms, direct messaging, and live office hours features. This tool allows instructors to make group projects significantly easier by forming groups in the application itself. While similar to Piazza and CometContact, Campuswire is primarily treated as a course management software that is more for the instructors than it is for students. The goal of CometContact is to be a student driven communication tool, not a heavily moderated and instructor managed application.

Lastly, the most similar project to what we aim to achieve with CometContact is CircleIn, which is a virtual student community where students in a class can make study groups, share resources, and build a more connected student community. CircleIn captures the spirit of

maintaining and improving the community among students, but lacks a structured collaboration system and the university systems connectivity we would be implementing.

7. Conclusion

Throughout the semester project, our team was able to deliver a structured, well-planned software proposal that aligned with our original vision quite well. However, certain adjustments needed to be made as we progressed and refined our understanding of the project. The majority of our changes revolved around clarifying the scope of the project. For example, we had to adjust our proposal around how our software would rely on existing UTD systems and databases. These changes were justified because they allowed for us to produce a more realistic proposal for our software in the end. Later on, by incorporating incremental development and accounting for scalability we were able to ensure that our final proposal met our standards. Overall we believe the project went well this semester and much was learned through the process.

8. References (If Applicable)

[1]“Software Engineer Salary in Dallas, TX,” www.indeed.com.

<https://www.indeed.com/career/software-engineer/salaries/Dallas--TX>